AtlasOps

Can 4 AI agents replace an on-call SRE team?

AMD Developer Hackathon 2026

> Real GKE cluster · Real Chaos Mesh · Real Prometheus alerts · **AMD MI300X**

Hari Kishanth — Da Big Three — St. Joseph's College of Engineering

The Problem

A P1 alert fires at 2 AM. The on-call SRE has 15 minutes to:

1. **Triage** — What's broken? How bad? Who's affected?
2. **Diagnose** — Why? Find root cause in 20k metrics
3. **Remediate** — Fix it without making it worse
4. **Communicate** — Slack update, postmortem, status page

Senior SRE average time: ~25 minutes

What if 4 specialized AI agents handled all 4 phases in parallel?

Architecture: 4 Agents, 20 Real Tools

```
Alertmanager → Coordinator → Triage Agent
                           → Diagnosis Agent   ← Prometheus + Jaeger
                           → Remediation Agent ← kubectl + Argo CD
                           → Comms Agent       ← Slack + Postmortem
```

Running on AMD MI300X (192 GB HBM3)

5 Qwen2.5 models co-hosted via vLLM:

- Triage, Diagnosis, Remediation, Comms: `Qwen2.5-7B + LoRA`
- LLM Judge: `Qwen2.5-72B` (adversarial scenario designer)

Real Infrastructure — Not Simulated

Layer	What's Real
Cluster	GKE Standard (us-central1, 3× e2-standard-4)
App	Google Online Boutique (11 microservices, gRPC)
Chaos	Chaos Mesh: PodChaos, NetworkChaos, StressChaos, DNSChaos
Metrics	Prometheus + Grafana (real scraping)
Traces	Jaeger + OpenTelemetry
GitOps	Argo CD (real rollbacks)
Alerts	Alertmanager → AtlasOps webhook (live)
DB	Cloud SQL Postgres 15

No mocks. No Docker Compose pretending to be cloud.

20 Real SRE Tools

```
kubectl_get  kubectl_logs  kubectl_describe  kubectl_top_pods
kubectl_rollout  kubectl_scale  kubectl_exec

promql_query  promql_query_range

jaeger_search  jaeger_get_trace

argocd_list_apps  argocd_app_history  argocd_rollback

gcloud_logs_read  cloud_monitoring_query

alertmanager_silence  alertmanager_list_alerts

slack_post_update  postmortem_draft
```

kube-sre-gym has 7. AtlasOps has 20.

Benchmark Results (Real GKE, May 9 2026)

Scenario	Outcome	Time	Score
Cloudflare 2019 (CPU saturation)	resolved	102.8s	0.856
GitHub 2018 (DB failover loop)	unresolved	35.5s	0.548
sf-001 (OOMKill crash loop)	partial	38.3s	0.722
Average	66% resolved	58.9s	0.709

Senior SRE with runbook: ~25 min average

AtlasOps: ~59 seconds average

Training Pipeline: SFT → GRPO on AMD MI300X

```
# SFT: QLoRA 4-bit + LoRA r=16
python training/sft.py \
  --model Qwen/Qwen2.5-7B-Instruct \
  --data data/sft_corpus.jsonl \
  --rocm  # AMD ROCm backend

# GRPO: Online RL against live GKE cluster
python training/grpo.py \
  --model checkpoints/sft_v3 \
  --loss_type dapo  # DAPO loss for stability
  --rocm
```

Curriculum: Spaced repetition [3,6,12,24,48h], mastery decay=0.85

Reward: 70% episode contract + 30% dense step rewards

Safety Architecture

```
Every mutating action requires approval:
  P0 incident → human must approve (Slack button)
  P1 incident → 60-second auto-approve window
  P2/P3       → automatic

Circuit Breaker:
  Max 50 tool calls per incident
  Max 10 mutations per hour
  Halts on repeated identical calls

Audit Log: HMAC hash-chained, append-only
Incident Correlator: 5-min dedup window
```

No agent can directly delete namespaces or stop cluster components.

The Cloudflare 2019 Replay (Live Demo)

```
16:36:06  [TRIAGE]      alertmanager: 3 active alerts found
16:36:12  [TRIAGE]      kubectl top: frontend CPU at 94%
16:36:16  [TRIAGE]      promql: error rate rising — P2 assigned
16:36:21  [DIAGNOSIS]   promql: container_cpu_usage peak confirmed
16:36:28  [DIAGNOSIS]   jaeger: frontend → adservice spans timing out
16:36:36  [DIAGNOSIS]   kubectl logs: regex CPU hog in ad classifier
16:36:37  [REMEDIATION] approval gate: P2 — auto-approved in 60s
16:38:17  [REMEDIATION] kubectl scale adservice 0→1: scale applied
16:38:21  [COMMS]       slack: [P2] incident update posted
16:38:28  [COMMS]       postmortem: saved (3 action items)

Total: 2 min 22 sec
```

Why AMD MI300X?

Feature	AMD MI300X	Benefit
192 GB HBM3	Fit 5 Qwen models simultaneously	No model swapping latency
ROCm + vLLM	Full LLM inference stack	Same code as NVIDIA H100
DAPO + QLoRA	Stable training on sparse SRE rewards	Less mode collapse
AMD AI Developer Program	Access for open-source AI research	AMD community

5 models co-hosted = 4 agent roles + 72B judge running concurrently

No token switching overhead = real-time incident response

What's Next

1. **Full AMD MI300X training** — SFT on 5k trajectories + GRPO RL
2. **Expected: +20pp improvement** after training (0.709 → ~0.90+)
3. **Multi-tenancy** — multiple teams, cost allocation, RBAC
4. **Production hardening** — mTLS, audit export, PagerDuty integration
5. **Open-source release** — Apache 2.0, Helm chart, OOTB Alertmanager config

Thank You

AtlasOps — Multi-agent SRE on AMD MI300X

GitHub: `github.com/Harikishanth/AtlasOps`
HF Space: `huggingface.co/spaces/DarDrax/atlasops`
Demo: `http://34.132.118.204` (Online Boutique live)

Built in 72 hours by Hari Kishanth
AMD Developer Hackathon 2026 — lablab.ai